

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. What is the output?

```
def fun():
    try:
        return 10
    finally:
        return 20
```

```
print(fun())
```

- A)20
- B)None
- C)RuntimeError
- D)10

**Q2. if test.py has
print("A")**

```
if __name__ == "__main__":
    print("B")
```

**If test.py is executed directly, what
will be printed?**

- A)B
- B)A followed by B
- C)Nothing
- D)A

Q3. What is the output?

```
def fun(x):
    x += 10
    return x
```

```
x = 5
y = fun(x)
```

```
print(x, y)
```

- A)15 5
- B)5 5
- C)15 15
- D)5 15

**Q4. What happens when this
function is called?**

```
def fun(a=10, b):
```

```
    return a + b
```

- A)SyntaxError
- B)NameError
- C)TypeError
- D)It executes correctly

Q5. What is the output?

```
try:
    raise ValueError
except ValueError:
    print("A")
    raise
```

- A)No Output
- B)A, followed by propagation of the same exception
- C)B
- D)A, and execution ends normally

Q6. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

- A)36
- B)6
- C)6.0
- D)NameError

**Q7. What is the purpose of this
statement?**

```
if __name__ == "__main__":
```

- A)It prevents all imports
- B)It creates the main function
- C)It identifies code intended to run when the file is executed directly
- D)It prevents syntax errors

**Q8. What is the major difference
between the following?**

```
import math
```

```
and
```



from math import sqrt

- A)Both import only sqrt
- B)There is no difference
- C)First imports only sqrt
- D)First provides the module namespace; second imports sqrt directly

Q9. What does the following function return?

```
def fun():  
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

- A)None
- B)Hello None
- C)Hello
- D>Error

Q10. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

- A)The local file can shadow the intended standard-library module
- B)Python will always reject it
- C)Python automatically renames the local file
- D)random cannot be imported

Q11. What will happen?

```
def fun(n):  
    if n == 0:  
        return 1  
    return n * fun(n)
```

```
print(fun(3))
```

- A)6
- B)3
- C)RecursionError
- D)TypeError

Q12. What will be printed?

```
x = 10
```

```
def outer():
```

```
x = 20
```

```
def inner():  
    print(x)
```

```
x = 30  
inner()
```

```
outer()
```

- A)30
- B)20
- C)NameError
- D)10

Q13. What happens when this code runs?

```
def first():  
    second()
```

```
def second():  
    third()
```

```
def third():  
    raise RuntimeError("Failure")
```

```
try:  
    first()  
except RuntimeError:  
    print("Handled")
```

- A)NameError
- B)Handled
- C)TypeError
- D)RuntimeError cannot propagate through functions

Q14. What will normally happen?

```
import math  
import math
```

- A)The module is normally loaded once and reused
- B)The module is normally loaded and executed twice
- C)ImportError
- D)SyntaxError

Q15. What will be the output?

```
def fun(x):  
    try:
```



```

if x < 0:
    raise ValueError("negative")
return x
except ValueError:
    return 100
finally:
    print("cleanup")

```

print(fun(-5))

- A)cleanup
- 100
- B)negative
- cleanup
- C)100
- cleanup
- D)cleanup
- 5

Q16. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(
    lambda a, b: a * b,
    numbers
)
```

print(result)

- A)24
- B)0
- C)10
- D)9

Q17. What is the output?

```
f = lambda x: x + 5
```

print(f(10))

- A)15
- B)5
- C)10
- D>Error

Q18. What will be printed?

```
try:
    x = int("10")
except ValueError:
    print("A")

```

```

else:
    print("B")
finally:
    print("C")

```

- A)B
- B)B C
- C)A
- D)A C

Q19. What will be printed?

```

try:
    print(10 / 0)
except ZeroDivisionError:
    print("A")
except Exception:
    print("B")

```

- A)A
- B)A B
- C)B
- D)Nothing

Q20. What will be printed?

```

def fun():
    try:
        return 10
    finally:
        print("Finally")

```

print(fun())

- A)Finally only
- B)10 only
- C)10 followed by Finally
- D)Finally followed by 10

Q21. What is the output?

```

try:
    print(10 / 0)
except Exception:
    print("A")
except ZeroDivisionError:
    print("B")

```

- A)B
- B)SyntaxError
- C)A
- D)A B

Q22. What happens here?

```
def fun(a):
```



return a

print(fun())

- A)0
- B)NameError
- C)None
- D)TypeError

Q23. Why is

try:

print(10 / 0)

except Exception:

print("A")

except ZeroDivisionError:

print("B")

codes's second handler never reached?

- A)except blocks execute randomly
- B)ZeroDivisionError occurs before Exception
- C)Exception is a superclass of ZeroDivisionError
- D)ZeroDivisionError is not an exception

Q24. What will be the output?

x = 10

def outer():

x = 20

def inner():

global x

x = 30

inner()

print(x)

outer()

print(x)

- A)30
- 30
- B)20
- 30
- C)20
- 10
- D)30
- 20

Q25. Suppose sample.py contains:

x = 10

def display():

print("Hello")

What will this print?

import sample

print(sample.x)

sample.display()

- A>Error
- B)x
- display
- C)10
- Hello
- D)Only Hello

Q26. What is the output?

def outer():

x = 10

def inner():

return x + 5

return inner

f = outer()

print(f())

- A)None
- B)NameError
- C)15
- D)10

Q27. What will happen?

def outer():

x = 10

def inner():

nonlocal y

y = 20

inner()

outer()

- A)NameError
- B)20
- C)SyntaxError



D)10

Q28. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():  
    raise MyError("Something went  
wrong")
```

```
try:  
    test()  
except MyError as e:  
    print(type(e).__name__)
```

- A)MyError
- B)Something went wrong
- C)Exception
- D)TypeError

Q29. What happens?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, a=20))
```

- A)SyntaxError
- B)30
- C)TypeError
- D)20

Q30. What will be printed?

```
def fun(x=[]):  
    x.append(10)  
    return x
```

```
a = fun()  
b = fun()
```

```
print(a)  
print(b)
```

- A)[10, 10]
- [10]
- B)[10]
- [10]
- C>Error
- D)[10]
- [10, 10]

Q31. What happens here?

try:

```
    raise ValueError("Wrong value")  
except ValueError as e:  
    print(e)
```

- A)Wrong Value
- B)e
- C)Nothing
- D)ValueError

Q32. What is the output?

```
def fun(a=10, b=20):  
    return a - b
```

```
print(fun(30))
```

- A)30
- B)10
- C)-10
- D)20

Q33. What is the output?

```
def fun(x):  
    if x == 0:  
        return  
    return x + fun(x - 1)
```

```
print(fun(3))
```

- A)3
- B)None
- C)5
- D)6

Q34. What will happen?

```
def fun(a, b=10):  
    return a + b
```

```
print(fun(b=5, a=20))
```

- A)SyntaxError
- B)TypeError
- C)15
- D)25

Q35. What happens when this code executes?

```
def fun():  
    print(variable)  
    variable = 50
```

```
variable = 10
```



fun()

A)10

B)UnboundLocalError

C)NameError

D)50

Q36. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":  
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

A)A and B

B)Nothing

C)B

D)A

Q37. What is the output?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        nonlocal x
```

```
        x = x + 5
```

```
        return x
```

```
    inner()
```

```
    return x
```

```
print(outer())
```

A)15

B)UnboundLocalError

C)10

D)5

Q38. What is the output?

```
try:
```

```
    x = int("abc")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
print("C")
```

A)A C

B)B C

C)A

D)B

Q39. What will be the output?

```
x = 10
```

```
def fun():
```

```
    print(x)
```

```
    x = 20
```

```
fun()
```

A)20

B)10

C)NameError

D)UnboundLocalError

Q40. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a
```

```
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

A)10

10

B)ImportError

C)Module loaded

10

D)Module loaded

Module loaded

10

Q41. What is the output?

```
def fun(x):
```

```
    return x * 2
```

```
a = fun
```



print(a(5))

A)func(5)

B)5

C)10

D>Error

Q42. What will be printed?

```
def make_counter():
```

```
    count = 0
```

```
    def counter():
```

```
        nonlocal count
```

```
        count += 1
```

```
        return count
```

```
    return counter
```

```
a = make_counter()
```

```
b = make_counter()
```

```
print(a())
```

```
print(a())
```

```
print(b())
```

```
print(a())
```

A)1

2

3

4

B)1

1

1

1

C)1

2

1

3

D)0

1

0

2

Q43. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

A)sqrt(25)

B)5.0

C)5

D)25

Q44. Which object keeps track of modules that have already been imported during the current Python process?

A)sys.modules

B)sys.imports

C).__module__

D)sys.path

Q45. What will be printed?

```
def fun(a, b):
```

```
    return a + b
```

```
print(fun(10, b=20))
```

A)30

B)20

C)TypeError

D)SyntaxError

Q46. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

A)None

B)NameError

C)20

D)10

Q47. What will be printed?

```
def fun():
```

```
    print("A")
```

```
    return
```

```
    print("B")
```

```
fun()
```

A)A

B)Nothing

C)A B

D)B

Q48. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```



```

result = list(
    map(
        lambda x: x * 2,
        filter(lambda x: x % 2 == 0,
numbers)
    )
)

```

print(result)

A)[2, 4, 6, 8, 10]

B)[2, 4]

C)[4, 8]

D)[1, 3, 5]

Q49. What is the output?

```

def fun():
    try:
        print("A")
        raise ValueError
    except ValueError:
        print("B")
        return 10
    else:
        print("C")
        return 20
    finally:
        print("D")

```

print(fun())

A)A

B

10

D

B)A

C

D

20

C)A

B

C

D

10

D)A

B

D

10

Q50. What will be printed?

```

class InvalidAgeError(Exception):
    pass

```

```

def check_age(age):
    if age < 18:
        raise InvalidAgeError("Age is
too low")
    return "Allowed"

```

```

try:
    print(check_age(15))
except InvalidAgeError as e:
    print(type(e).__name__)
    print(e)

```

A)Nothing, because custom exceptions cannot be caught

B)InvalidAgeError

15

C)InvalidAgeError

Age is too low

D)Exception

Age is too low

